CESI
ÉCOLE D'INGÉNIEURS

Stockage & Big Data

Workshop | ETL avec Talend

Mercredi 21 octobre 2026

Mazou Marine
Fisa Info 25–28
CESI

TABLE DES MATIÈRES

Introduction **2**

Partie 1 : prise en main de Talend **3**
1. Le job . . . 3
2. Génération des données . . . 3
3. Transformations du tMap . . . 4
4. Fichiers de sortie et résultat . . . 4

Partie 2 : ETL des données de ventes **5**
1. Analyse des sources . . . 5
2. Unifier les dates . . . 6
3. Récupérer tous les enregistrements . . . 6
4. Chargement dans PostgreSQL . . . 7

Partie 3 : modèle en étoile dans HDFS **8**
1. Besoin et analyse des sources . . . 8
2. Schéma décisionnel . . . 8
3. Alimenter les dimensions . . . 9
4. Créer la table de faits . . . 11
5. Vérification dans Cloudera . . . 11

Conclusion **13**

INTRODUCTION

Ce workshop met en pratique l'ETL (*Extract, Transform, Load*) avec **Talend Open Studio for Big Data** : extraire des données hétérogènes, les nettoyer et les transformer, puis les charger dans une cible. Il comporte trois parties :

1. la prise en main de Talend sur des données générées (génération, tri, filtrage, export Excel et XML) ;
2. un ETL complet sur les données de ventes d'un client, de trois fichiers CSV vers une base PostgreSQL ;
3. la conception d'un modèle en étoile et son alimentation dans un Data Lake HDFS, sur la machine Cloudera.

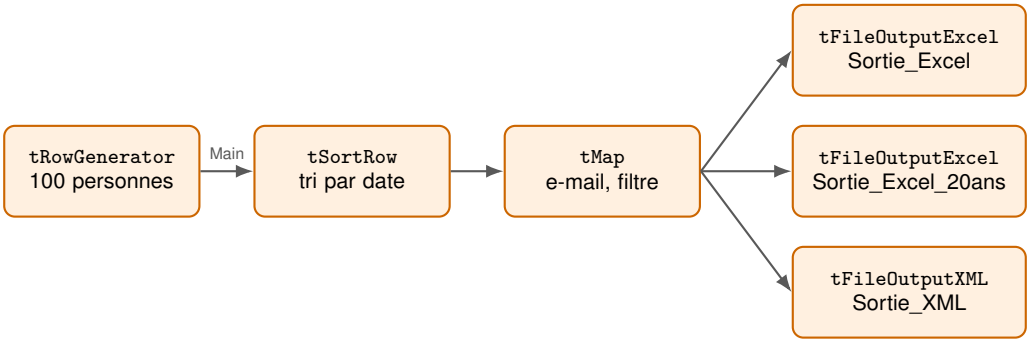
Les jeux de données des parties 2 et 3 ont été analysés avant de construire les jobs. Les anomalies relevées et les chiffres attendus sont donnés à chaque étape pour vérifier le résultat des jobs. Chaque partie correspond à un job du projet Talend : `Job_partie1`, `Job_partie2_ventes`, puis `Job_partie3_dimensions` et `Job_partie3_faits`.

Composant	Rôle
`tRowGenerator`	Générer des lignes de test selon un schéma et des fonctions
`tSortRow`	Trier les lignes
`tMap`	Transformer, filtrer, joindre et répartir les lignes vers plusieurs sorties
`tFileInputDelimited`	Lire un fichier délimité (CSV)
`tUniqRow`	Supprimer les doublons sur une clé
`tAggregateRow`	Regrouper et calculer des agrégats (somme, maximum)
`tFileOutputExcel`, `tFileOutputXML`	Écrire un fichier Excel ou XML
`tDBOutput` **(PostgreSQL)**	Créer et alimenter une table PostgreSQL
`tHDFSOutput`, `tHDFSInput`	Écrire et lire un fichier dans HDFS

PARTIE 1 : PRISE EN MAIN DE TALEND

1. LE JOB

Le job génère des personnes, les trie par date de naissance, puis les répartit vers trois fichiers : toutes les personnes en Excel et en XML, et les seules personnes de 20 ans ou plus dans un second fichier Excel.



2. GÉNÉRATION DES DONNÉES

Le schéma du `tRowGenerator` (*Edit schema*) et la fonction de génération de chaque colonne :

Colonne	Type	**Fonction**
Id	Integer	`Numeric.sequence("s1", 1, 1)`
Nom	String	`TalendDataGenerator.getLastName()`
Prenom	String	`TalendDataGenerator.getFirstName()`
Date	Date	`TalendDate.getRandomDate("2000-01-01", "2010-12-31")`
Ville	String	`TalendDataGenerator.getUsCity()`

`Numeric.sequence` numérote les lignes à partir de 1. Le motif de la colonne `Date` est `"dd-MM-yyyy"` : c'est le format d'affichage dans les fichiers de sortie. Le `tSortRow` trie ensuite sur `Date`, en ordre croissant et en mode *date*.

3. TRANSFORMATIONS DU TMAP

Le `tMap` reçoit le flux trié à gauche et alimente trois tables de sortie à droite. Les trois sorties reprennent toutes les colonnes d'entrée et ajoutent une colonne `Email` :

Colonne Email (les trois sorties)

```
row5.Prenom + "." + row5.Nom + "@cesi.fr"
```

`row5` est le nom du flux entrant dans le `tMap` : il change d'un job à l'autre et doit être adapté. On peut ajouter `.toLowerCase()` pour obtenir des adresses en minuscules (`claire.gute@cesi.fr`).

La sortie `Sortie_Excel_20ans` reçoit en plus un **filtre** (*Expression Filter*) :

Filtre du sujet : âge approché en jours

```
TalendDate.diffDate(TalendDate.getCurrentDate(), row5.Date) / 365 >= 20
```

Cette formule divise un nombre de jours par 365 et ignore les années bissextiles : une personne qui fête ses 20 ans dans les jours qui viennent est déjà comptée. Le test exact compare la date des 20 ans à la date du jour :

Filtre exact : la personne a eu 20 ans aujourd'hui ou avant

```
TalendDate.addDate(row5.Date, 20, "yyyy").compareTo(TalendDate.getCurrentDate()) <= 0
```

4. FICHIERS DE SORTIE ET RÉSULTAT

Les trois fichiers sont écrits dans `C:/Talend/WS2/sorties/` :

Composant	Fichier	Paramètres
`tFileOutputExcel`	`Sortie_Excel.xlsx`	Format 2007 (`.xlsx`), *Inclure l'en-tête*
`tFileOutputExcel`	`Sortie_Excel_20ans.xlsx`	Idem, seules les lignes qui passent le filtre
`tFileOutputXML`	`Sortie_XML.xml`	Balise racine `root`, une balise `row` par personne

Le sujet écrit `.xslx` : l'extension correcte est `.xlsx`. À l'exécution, les 100 lignes générées traversent le job. Nées entre 2000 et 2010, les personnes ont de 15 à 26 ans au 22 octobre 2026 : lors de notre exécution, **68** passent le filtre (le tirage est aléatoire). Elles sont exactement celles du fichier complet nées avant le 22/10/2006, ce qui valide le filtre. Les deux autres fichiers contiennent les 100 lignes, triées du 01/04/2000 au 20/10/2010.

PARTIE 2 : ETL DES DONNÉES DE VENTES

1. ANALYSE DES SOURCES

Le client fournit trois fichiers CSV (séparateur virgule, en-tête sur la première ligne) :

Fichier	Lignes	Colonnes
`Features data set.csv`	8 190	Store, Date, Temperature, Fuel_Price, MarkDown1 à 5, CPI, Unemployment, IsHoliday
`sales data-set.csv`	421 570	Store, Dept, Date, Weekly_Sales, IsHoliday
`stores data-set.csv`	48	Store, Type, Size

Le client signale des factures erronées, des doublons et des enregistrements vides. L'analyse des fichiers les localise précisément :

Anomalie	Constat	Traitement
Dates hétérogènes	8 189 dates en `dd/MM/yyyy`, une seule en `dd-MM-yyyy` : la dernière ligne de Features (magasin 45, `26-07-2013`)	Unifier le format dans le `tMap`
Valeurs `NA`	585 lignes de Features sans CPI ni Unemployment (mai à juillet 2013) ; MarkDown1 à 5 absents sur 51 à 64 % des lignes	Convertir `NA` en `NULL`
Doublons	Stores : 48 lignes pour 45 magasins (magasins 1 et 2 répétés)	`tUniqRow` sur `Store`
Factures erronées	Sales : 1 285 ventes hebdomadaires négatives (jusqu'à $-4\,988,94$)	Écarter vers une table de rejets

Les 73 ventes à zéro sont conservées : une semaine sans vente dans un rayon est une information valide. Les MarkDown absents aussi : ils signifient qu'aucune démarque n'a eu lieu cette semaine-là.

2. UNIFIER LES DATES

Que remarque-t-on ? Dans Features, toutes les dates sont au format `dd/MM/yyyy` sauf la dernière, écrite avec des tirets (`26-07-2013`). Si la colonne est lue comme une date de motif `dd/MM/yyyy`, cette ligne ne peut pas être convertie.

La solution est de lire la colonne `Date` comme une chaîne (`String`) dans les métadonnées du fichier, puis de la convertir dans le `tMap` après avoir remplacé les tirets par des barres obliques :

Expression de la colonne Date (sortie de type Date)

```
TalendDate.parseDate("dd/MM/yyyy", row1.Date.replace("-", "/"))
```

Les deux formats deviennent ainsi une même date. Le même traitement est appliqué au fichier Sales par précaution, même si toutes ses dates sont déjà au bon format.

3. RÉCUPÉRER TOUS LES ENREGISTREMENTS

A-t-on récupéré l'ensemble des enregistrements ? Non. L'assistant de métadonnées devine le type de chaque colonne à partir des premières lignes du fichier : CPI et Unemployment y sont numériques, et sont donc proposés en `Float`. Les 585 lignes où ces colonnes valent `NA`, dont la ligne à la date mal formée, ne peuvent pas être converties et sont écartées à la lecture : le `tLogRow` n'affiche alors que 7 605 lignes sur 8 190.

Comment le résoudre ? Il faut lire ces colonnes comme du texte, puis faire la conversion soi-même :

1. dans les métadonnées de Features, passer `Date`, `CPI`, `Unemployment` et `MarkDown1` à `MarkDown5` en `String` ;
2. dans le `tMap`, convertir chaque colonne numérique en traitant `NA` comme une valeur absente :

Expression de la colonne CPI (sortie Double nullable)

```
row1.CPI.trim().equals("NA") ?  null :  Double.parseDouble(row1.CPI.trim())
```

La même expression est appliquée à Temperature, Fuel_Price, Unemployment et aux cinq MarkDown. Le type `Double` est préféré à `Float`, qui arrondirait le CPI (`211.0963582`) à sept chiffres significatifs. Le `tLogRow` affiche alors les **8 190 lignes**. On peut aussi relier la sortie *Rejects* du `tFileInputDelimited` à un `tLogRow` pour voir les lignes écartées et le message d'erreur de chaque ligne.

4. CHARGEMENT DANS POSTGRESQL

Dans pgAdmin 4, on crée la base `Ventes`. Dans Talend, la connexion est créée dans le Référentiel (*Connexions aux bases de données → Créer une connexion*, type PostgreSQL), testée, puis exportée en **contexte** : hôte, port, base, utilisateur et mot de passe deviennent des variables réutilisables, au lieu d'être écrits dans chaque composant. Dans le job `Job_partie2_ventes`, ce sont les variables `pg_host`, `pg_port`, `pg_base`, `pg_schema`, `pg_user` et `pg_password`, utilisées par les quatre `tDBOutput`.

Chaque fichier est chargé par un flux de ce type :

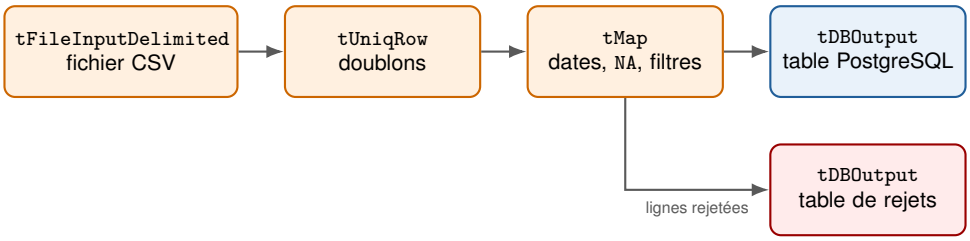


Table	Traitement dans le job	Lignes attendues
`features`	Dates unifiées, `NA` convertis en `NULL`	8 190
`stores`	`tUniqRow` sur `Store`	45
`sales`	Filtre `row1.Weekly_Sales >= 0` sur la sortie principale	420 285
`sales_rejets`	Filtre inverse `row1.Weekly_Sales < 0`	1 285

Dans chaque `tDBOutput`, la table est *supprimée si elle existe puis créée*, et les données *insérées* : le job peut être relancé sans doublons. Le sujet demande de passer la longueur de `Unemployment` de 5 à 11 : la longueur devinée par l'assistant est trop courte (`13.975`) ; en lisant la colonne comme texte puis en la convertissant en `Double`, le problème ne se pose plus. Les ventes négatives sont écartées et non supprimées : ce sont peut-être des retours de marchandise, et seul le client peut le dire. Dans pgAdmin, une requête vérifie les volumes chargés :

```
SELECT 'features' AS table_chargee, COUNT(*) FROM features
UNION ALL SELECT 'stores', COUNT(*) FROM stores
UNION ALL SELECT 'sales', COUNT(*) FROM sales
UNION ALL SELECT 'sales_rejets', COUNT(*) FROM sales_rejets;
```

PARTIE 3 : MODÈLE EN ÉTOILE DANS HDFS

1. BESOIN ET ANALYSE DES SOURCES

Le second client veut le **chiffre d'affaires par produit et par client**, pour organiser ses produits par région, limiter les pertes et récompenser ses meilleurs clients. Il fournit deux fichiers (séparateur point-virgule, encodage ISO-8859-1), liés ligne à ligne par `Row_ID` :

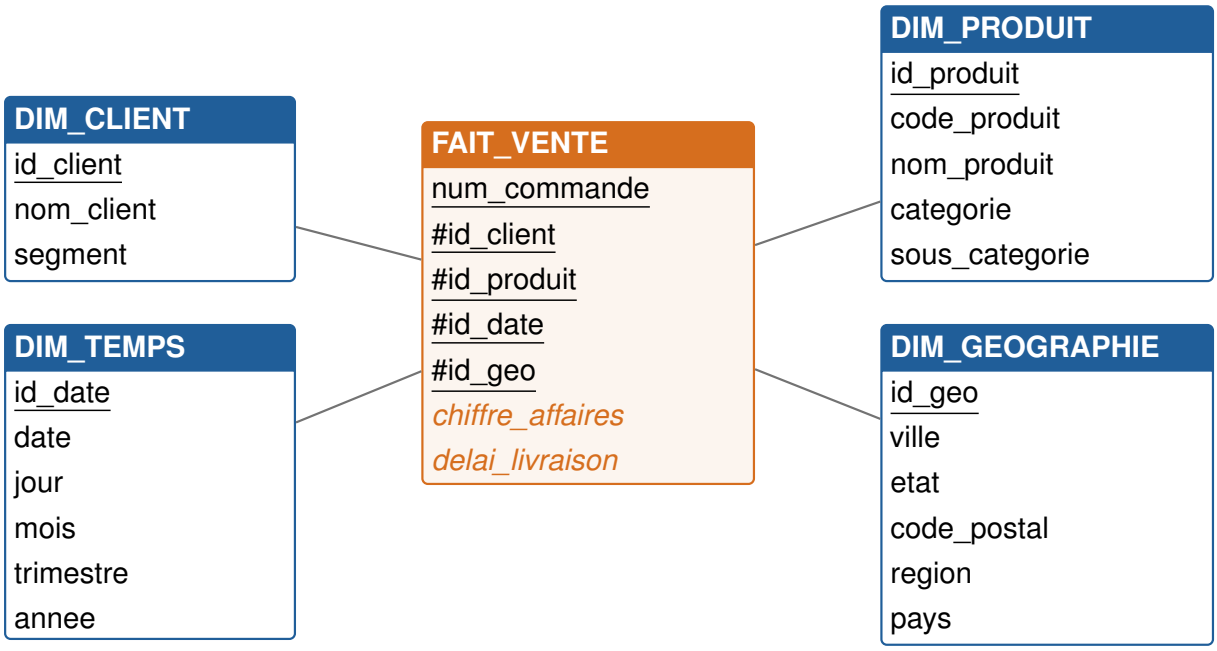
- `Customer.csv` : 9 800 lignes, une par ligne de commande (client, segment, ville, État, code postal, région) ;
- `Product.csv` : 9 800 lignes (commande, dates de commande et d'expédition, mode de livraison, produit, catégorie, montant `Sales`).

L'analyse révèle trois points qui orientent le modèle :

- **2 163 lignes de Product n'ont pas de montant**. Quand le nom du produit contenait un point-virgule, il a été placé entre guillemets et la colonne `Sales` a été perdue à l'export (`"Hon Deluxe Fabric Upholstered Stacking Chairs; Rounded Back"` sans montant). Ces lignes ne peuvent pas entrer dans le calcul du CA : elles sont écartées et signalées au client ;
- **un client a plusieurs adresses** : 779 des 793 clients ont été livrés dans plusieurs villes. La ville n'est donc pas un attribut du client mais du lieu de livraison, d'où une dimension Géographie séparée ;
- **un code produit peut porter plusieurs noms** : 1 861 codes pour 1 893 couples (code, nom). La dimension Produit reçoit donc une clé de substitution par couple.

2. SCHÉMA DÉCISIONNEL

Les données viennent d'un seul métier, la vente : un **modèle en étoile** suffit. Le grain est **un produit dans une commande**, et la mesure principale est le chiffre d'affaires.



Légende : clé primaire #clé étrangère *mesure*

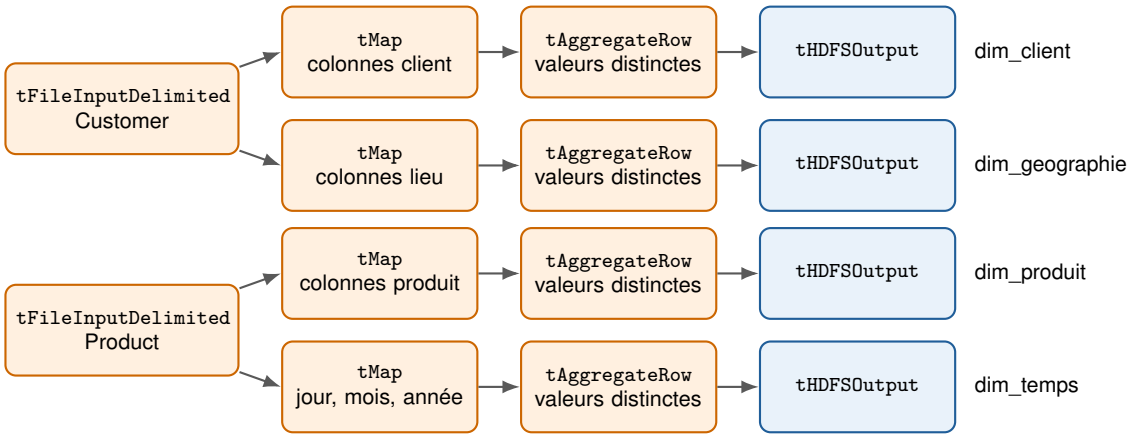
Table	Source et clé	Lignes attendues
`dim_client`	Customer_ID, nom, segment	793
`dim_produit`	Couples (Product_ID, nom), clé de substitution	1 893
`dim_geographie`	Couples (ville, État, code postal), clé de substitution	628
`dim_temps`	Dates de commande distinctes, clé `AAAAMMJJ`	1 230
`fait_vente`	Lignes avec un montant, regroupées par commande et par clés	7 632

`num_commande` est une dimension dégénérée : il identifie la commande sans avoir de table propre, et fait partie de la clé de la table de faits. Le délai de livraison (date d'expédition moins date de commande, en jours) est une mesure complémentaire, utile pour comparer les modes de livraison.

3. ALIMENTER LES DIMENSIONS

Les deux fichiers sont d'abord déclarés en métadonnées (*Fichier délimité*, séparateur `;`, encodage ISO-8859-1, en-tête d'une ligne). La connexion HDFS pointe sur la machine Cloudera (`hdfs://quickstart.cloudera:8020`, utilisateur `cloudera`, distribution Cloudera CDH 5.12) ; ces valeurs sont des variables de contexte, à modifier en un seul endroit si la machine a une autre adresse.

`Product.csv` est lu **sans** l'option CSV : avec elle, les guillemets jamais refermés de certains noms de produits feraient fusionner plusieurs lignes. Le schéma compte une onzième colonne, `Sales_extra` : quand un point-virgule coupe le nom du produit en deux, le montant glisse dans cette colonne et le `tMap` va l'y chercher. Un job alimente les quatre dimensions :



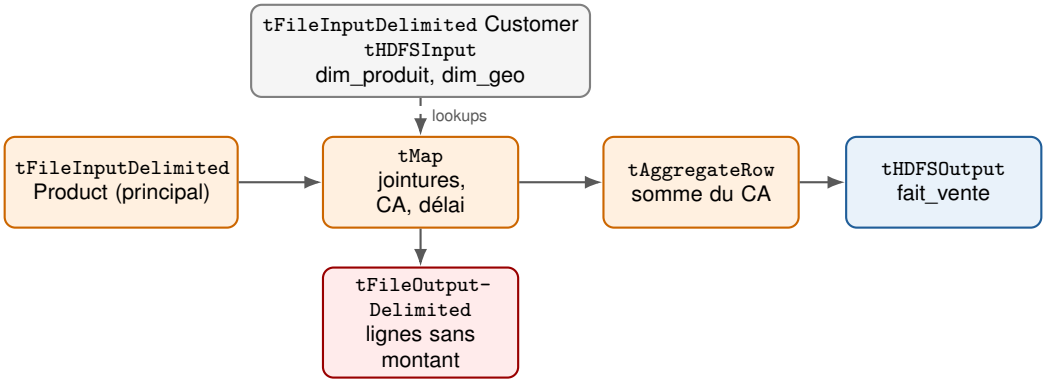
Chaque `tAggregateRow` regroupe sur les colonnes de la dimension (sans opération) : il ne garde qu'une ligne par valeur distincte. Un second `tMap` après l'agrégation ajoute la clé de substitution avec `Numeric.sequence("geo", 1, 1)` (une séquence par dimension). La dimension Temps se calcule à partir de `Order_Date`, lue comme une date de motif `dd/MM/yyyy` :

Colonne	Expression du tMap
`id_date`	`Integer.parseInt(TalendDate.formatDate("yyyyMMdd", row1.Order_Date))`
`jour`	`Integer.parseInt(TalendDate.formatDate("dd", row1.Order_Date))`
`mois`	`Integer.parseInt(TalendDate.formatDate("MM", row1.Order_Date))`
`trimestre`	`(Integer.parseInt(TalendDate.formatDate("MM", row1.Order_Date)) - 1) / 3 + 1`
`annee`	`Integer.parseInt(TalendDate.formatDate("yyyy", row1.Order_Date))`

Chaque `tHDFSOutput` écrit un fichier dans `/user/cloudera/datalake/` (séparateur `;`, avec en-tête). Pour pouvoir relancer le job, l'action passe de *Créer* à *Écraser* : sinon la deuxième exécution échoue car le fichier existe déjà.

4. CRÉER LA TABLE DE FAITS

Une fois les dimensions écrites, un second job construit la table de faits. Il relit les dimensions depuis HDFS pour retrouver leurs clés de substitution :



- **Jointure client** : Customer est joint à Product sur `Row_ID` ; on récupère `Customer_ID` et le lieu de livraison.
- **Clés de substitution** : `id_produit` est trouvé dans `dim_produit` sur le couple (code, nom), `id_geo` dans `dim_geographie` sur (ville, État, code postal). Les jointures sont en mode *Inner Join*, avec une sortie de rejet pour les lignes sans correspondance.
- **Mesures** : `chiffre_affaires` reprend `Sales` s'il est numérique, sinon `Sales_extra`, sinon `null` (variable `Var.ca` du `tMap`) ; `delai_livraison` vaut `TalendDate.diffDate(row1.Ship_Date, row1.Order_Date)`.
- **Filtre** : la sortie principale ne garde que `Var.ca != null` ; les 2 163 lignes sans montant partent dans `C:/Talend/WS2/sorties/lignes_sans_montant.csv`.

Le `tAggregateRow` regroupe sur la commande et les quatre clés, fait la somme du chiffre d'affaires et prend le maximum du délai. Cinq commandes contiennent deux fois le même produit pour le même client et le même lieu : leurs lignes sont fusionnées, et le fichier `fait_vente` contient **7 632 lignes** pour les 7 637 lignes avec montant.

5. VÉRIFICATION DANS CLOUDERA

Dans la machine Cloudera, les fichiers se vérifient en ligne de commande (le sujet écrit `dhfs`, la commande est bien `hdfs`) :

```
hdfs dfs -ls /user/cloudera/datalake/
hdfs dfs -cat /user/cloudera/datalake/fait_vente.csv | head -5
hdfs dfs -cat /user/cloudera/datalake/dim_client.csv | wc -l      # 794 avec l'en-tête
```

On peut aussi les parcourir dans **Hue** (*Navigateur de fichiers*). Pour répondre directement à la demande du client, on déclare les fichiers comme tables Hive externes (sans en-tête grâce à `skip.header.line.count`), puis on interroge l'étoile :

CA par région et par catégorie

```
SELECT g.region, p.categorie, ROUND(SUM(f.chiffre_affaires), 2) AS ca
FROM fait_vente f
JOIN dim_geographie g ON g.id_geo = f.id_geo
JOIN dim_produit  p ON p.id_produit = f.id_produit
GROUP BY g.region, p.categorie
ORDER BY ca DESC;
```

Les valeurs attendues, calculées sur les 7 637 lignes avec montant, servent à contrôler le chargement :

Région	CA	Catégorie	CA
West	599 187,75	Technology	799 304,53
East	538 805,83	Office Supplies	579 028,41
Central	408 773,37	Furniture	507 209,06
South	338 775,06		
Total	**1 885 542,01**	**Total**	**1 885 542,01**

Les trois meilleurs clients, à récompenser en priorité, sont Sean Miller (24 257,63), Tamara Chand (19 017,85) et Raymond Buch (15 117,34). Ces montants sont calculés hors des 2 163 lignes sans montant : le CA réel du client est donc plus élevé, et il faut lui demander un nouvel export de `Product.csv` pour le compléter.

CONCLUSION

Ce workshop a couvert la chaîne ETL complète avec Talend :

1. **extraire** : lire des fichiers délimités en précisant séparateur, encodage et types, en sachant que les types devinés par l'assistant peuvent faire perdre des lignes sans erreur visible ;
2. **transformer** : unifier les dates, convertir les valeurs `NA`, supprimer les doublons, écarter les lignes erronées vers des rejets plutôt que de les supprimer ;
3. **charger** : vers des fichiers Excel et XML, dans une base PostgreSQL, puis dans un Data Lake HDFS sous la forme d'un modèle en étoile (dimensions d'abord, faits ensuite).

Le `tMap` est le composant central : il réunit la transformation, le filtrage, les jointures et la répartition des lignes. Dans les deux jeux de données réels, l'analyse préalable a été décisive : une date mal formée, des valeurs `NA` ou un point-virgule dans un nom de produit suffisent à fausser les volumes chargés. Contrôler le nombre de lignes à chaque étape est le seul moyen de s'en apercevoir.